

Laporan Analisis: Tugas Hybrid Socket

Vicky Mahfudy 25/573019/PPA/07207

Bagian A: Implementasi Pemrograman Soket

A.1 Layanan TCP Multi-Client

Server (TCPServer.py) membuat 1 welcoming socket yang tetap terikat ke port 12000 seumur hidup proses, hanya dipakai untuk `accept()`. Setiap `accept()` yang berhasil menghasilkan connection socket baru, yang langsung diserahkan ke thread worker terpisah (`threading.Thread`) untuk melayani seluruh pertukaran data dengan client tersebut. Welcoming socket tidak pernah dipakai untuk kirim/terima data aplikasi dan hidup selama server berjalan, sedangkan connection socket dibuat per client, hanya hidup selama sesi client itu, dan ditutup begitu client disconnect. Karena kedua jenis socket ini terpisah, main thread bisa langsung `accept()` client baru tanpa menunggu client lama selesai, itulah dasar konkurensi.

Bukti: 3 client (A, B, C) dijalankan hampir bersamaan, connect-nya saling menyusul sebelum satu pun disconnect, dan Wireshark menunjukkan tiga three-way handshake terpisah dari tiga port ephemeral berbeda, semuanya menuju port 12000 yang sama.

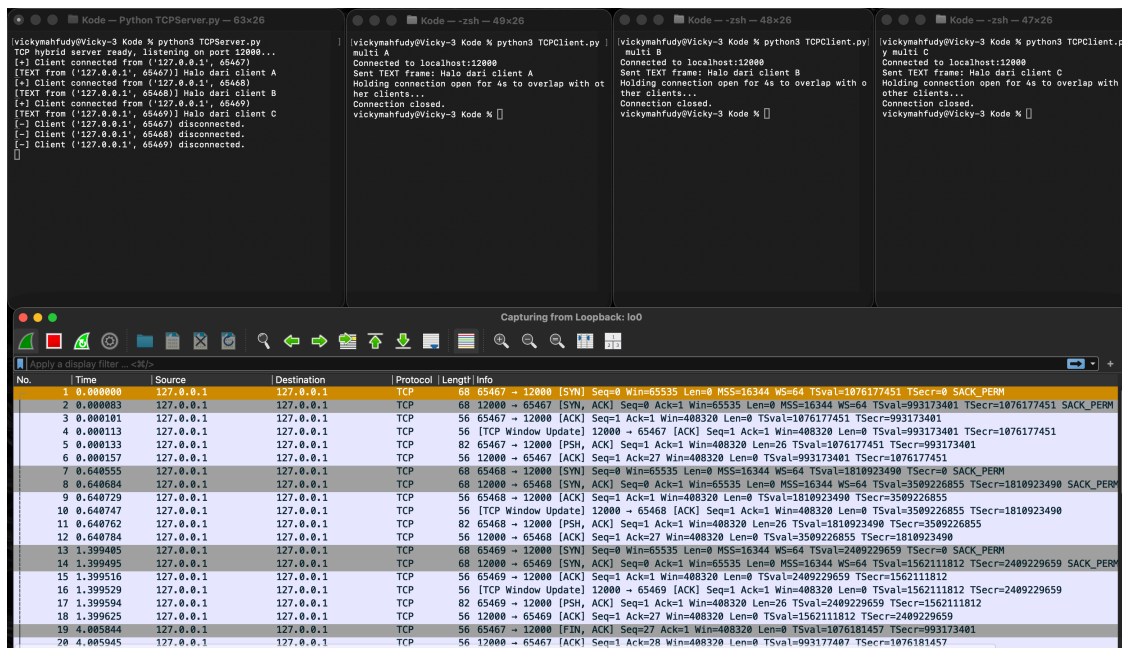
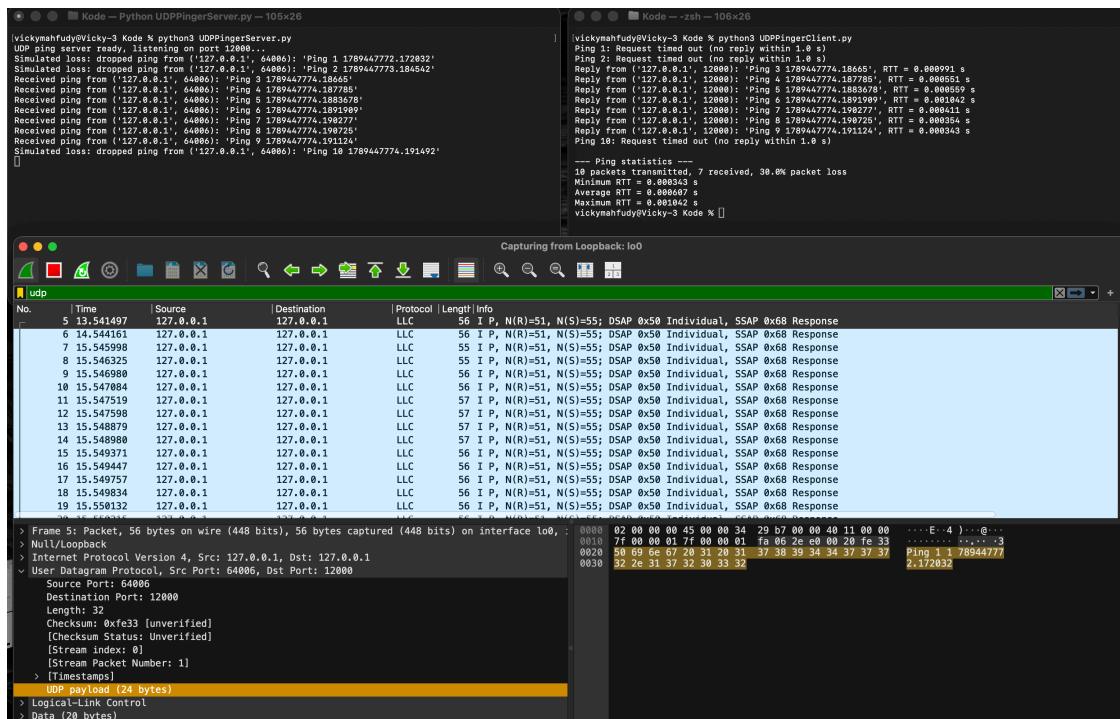


Figure 1: Bukti konkurensi TCP multi-client: 3 client terhubung bersamaan, log server dan tiga three-way handshake terpisah di Wireshark

A.2 Layanan UDP Pinger

Client (UDPPingerClient.py) memakai `clientSocket.setTimeout(1.0)` sebelum mengirim 10 ping berurutan. Jika balasan tidak datang dalam 1 detik, `exception timeout` ditangkap, ping tersebut dihitung hilang, dan loop lanjut ke ping berikutnya tanpa pernah blocking selamanya. Di akhir 10 kali kirim, client menghitung RTT minimum/rata-rata/maksimum dari ping yang berhasil, serta packet loss rate. Untuk mendemonstrasikan penanganan loss secara nyata, server (UDPPingerServer.py) sengaja men-drop ~30% ping yang masuk secara acak.

Hasil uji: ping 1, 2, dan 10 di-drop server sehingga client timeout tepat pada ketiganya; ping 3-9 berhasil dibalas dengan RTT sub-milidetik. Statistik akhir: 10 transmitted, 7 received, 30% packet loss.



The screenshot displays two terminal windows and a Wireshark packet capture. The left terminal shows the server (UDPPingerServer.py) listening on port 12000 and dropping ping 1, 2, and 10. The right terminal shows the client (UDPPingerClient.py) sending 10 pings, with ping 1, 2, and 10 timing out. The Wireshark interface shows a list of 19 UDP packets, with packet 5 selected. The packet details pane shows the structure of the UDP packet, including the payload (24 bytes).

```
vickymahfudy@Vicky-3 Kode % python3 UDPPingerServer.py
UDP ping server ready, listening on port 12000...
Simulated loss: dropped ping from ('127.0.0.1', 64006): 'Ping 1 1789447772.172032'
Simulated loss: dropped ping from ('127.0.0.1', 64006): 'Ping 2 1789447775.184542'
Received ping from ('127.0.0.1', 64006): 'Ping 3 1789447774.18665'
Received ping from ('127.0.0.1', 64006): 'Ping 4 1789447774.187785'
Received ping from ('127.0.0.1', 64006): 'Ping 5 1789447774.188378'
Received ping from ('127.0.0.1', 64006): 'Ping 6 1789447774.189199'
Received ping from ('127.0.0.1', 64006): 'Ping 7 1789447774.190277'
Received ping from ('127.0.0.1', 64006): 'Ping 8 1789447774.190725'
Received ping from ('127.0.0.1', 64006): 'Ping 9 1789447774.191124'
Simulated loss: dropped ping from ('127.0.0.1', 64006): 'Ping 10 1789447774.191492'

vickymahfudy@Vicky-3 Kode % python3 UDPPingerClient.py
Ping 1: Request timed out (no reply within 1.0 s)
Ping 2: Request timed out (no reply within 1.0 s)
Reply from ('127.0.0.1', 12000): 'Ping 3 1789447774.18665', RTT = 0.000991 s
Reply from ('127.0.0.1', 12000): 'Ping 4 1789447774.187785', RTT = 0.000551 s
Reply from ('127.0.0.1', 12000): 'Ping 5 1789447774.188378', RTT = 0.000859 s
Reply from ('127.0.0.1', 12000): 'Ping 6 1789447774.189199', RTT = 0.001042 s
Reply from ('127.0.0.1', 12000): 'Ping 7 1789447774.190277', RTT = 0.000411 s
Reply from ('127.0.0.1', 12000): 'Ping 8 1789447774.190725', RTT = 0.000334 s
Reply from ('127.0.0.1', 12000): 'Ping 9 1789447774.191124', RTT = 0.000343 s
Ping 10: Request timed out (no reply within 1.0 s)

--- Ping statistics ---
10 packets transmitted, 7 received, 30.0% packet loss
Minimum RTT = 0.000343 s
Average RTT = 0.000607 s
Maximum RTT = 0.001042 s
vickymahfudy@Vicky-3 Kode %
```

No.	Time	Source	Destination	Protocol	Length	Info
5	13.541497	127.0.0.1	127.0.0.1	LLC	56	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response
6	14.544161	127.0.0.1	127.0.0.1	LLC	56	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response
7	15.545998	127.0.0.1	127.0.0.1	LLC	55	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response
8	15.546325	127.0.0.1	127.0.0.1	LLC	55	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response
9	15.546980	127.0.0.1	127.0.0.1	LLC	56	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response
10	15.547084	127.0.0.1	127.0.0.1	LLC	56	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response
11	15.547519	127.0.0.1	127.0.0.1	LLC	57	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response
12	15.547598	127.0.0.1	127.0.0.1	LLC	57	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response
13	15.548079	127.0.0.1	127.0.0.1	LLC	57	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response
14	15.548900	127.0.0.1	127.0.0.1	LLC	57	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response
15	15.549371	127.0.0.1	127.0.0.1	LLC	56	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response
16	15.549447	127.0.0.1	127.0.0.1	LLC	56	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response
17	15.549757	127.0.0.1	127.0.0.1	LLC	56	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response
18	15.549834	127.0.0.1	127.0.0.1	LLC	56	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response
19	15.550132	127.0.0.1	127.0.0.1	LLC	56	I P, N(R)=51, N(S)=55; DSAP 0x50 Individual, SSAP 0x68 Response

```
> Frame 5: Packet, 56 bytes on wire (448 bits), 56 bytes captured (448 bits) on interface lo0, ...
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> User Datagram Protocol, Src Port: 64006, Dst Port: 12000
  Source Port: 64006
  Destination Port: 12000
  Length: 32
  Checksum: 0xfe33 [unverified]
  [Checksum Status: Unverified]
  [Stream index: 0]
  [Stream Packet Number: 1]
  [Timestamps]
  UDP payload (24 bytes)
  Logical-Link Control
  Data (20 bytes)
```

Figure 2: Bukti UDP Pinger: log server/client dan packet capture Wireshark datagram UDP request/reply pada port 12000

Bagian B: Analisis Mendalam & Arsitektur Jaringan

B.1 Byte-Stream vs Message Boundary

TCP memandang data sebagai aliran byte tanpa batas pesan: TCP boleh menggabungkan beberapa `send()` berurutan menjadi satu segmen atau memecah satu `send()` menjadi beberapa segmen, dan `recv()` tidak dijamin selaras satu-satu dengan `send()`. UDP sebaliknya menjaga message boundary: setiap `sendto()` = satu datagram utuh, setiap `recvfrom()` mengembalikan tepat satu datagram itu, karena UDP tidak punya abstraksi byte stream.

Dampak teknis: TCPCClient.py demo3 mengirim 3 pesan tanpa delay. Wireshark membuktikan hanya **2 segmen TCP** yang benar-benar terkirim: segmen pertama Len=21 (pesan pertama), segmen kedua Len=39 yaitu gabungan pesan kedua (19 byte) dan ketiga (20 byte) yang digabung

TCP menjadi satu segmen karena dikirim tanpa delay.

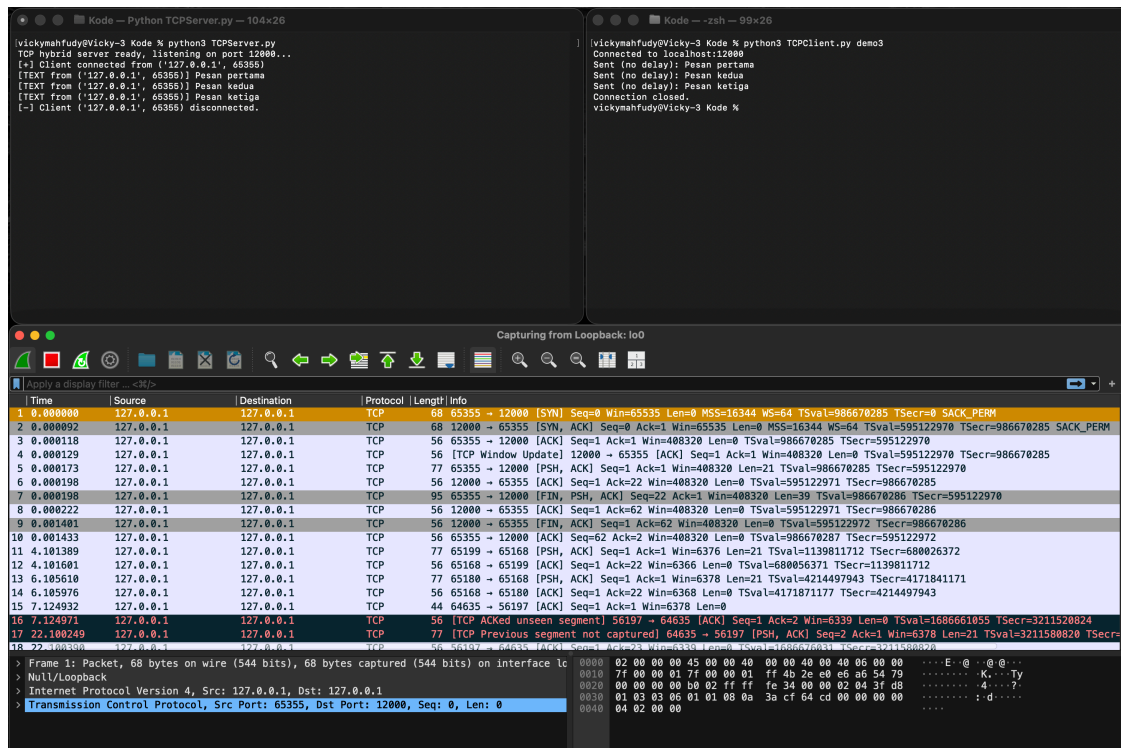


Figure 3: Bukti Wireshark: 3 pesan dikirim tanpa delay, hanya 2 segmen TCP terkirim di kabel

Solusi protokol aplikasi: setiap frame diawali 4-byte length prefix yang memberitahu penerima persis berapa byte payload yang harus dibaca. Fungsi `recv_exact()` terus membaca sampai jumlah byte lengkap, sehingga pemisahan pesan dilakukan di level aplikasi, bukan mengandalkan batas `recv()`. Terbukti server tetap menampilkan 3 pesan terpisah dengan benar meski di kabel hanya 2 segmen.

B.2 Analisis Skalabilitas Socket

Server TCP membuka $N + 1$ soket untuk N client: 1 welcoming socket tetap (port 12000) plus N connection socket (satu per client aktif, dibedakan lewat 4-tuple IP/port lokal-remote). Terbukti pada demo multi-client: 3 client = 3 connection socket + 1 welcoming socket = 4 soket total.

Server UDP hanya butuh **1 soket** karena connectionless, tidak ada `listen()/accept()/state` per sesi; `recvfrom()` sekaligus memberi alamat pengirim untuk dibalas langsung. Implikasi: server UDP memakai 1 FD dan 1 buffer kernel berapa pun banyak client, sedangkan server TCP memakai FD dan buffer yang bertumbuh $O(N)$, dan tiap connection socket mengonsumsi entry tabel koneksi kernel yang membatasi jumlah koneksi simultan secara praktis. Server UDP tidak punya batasan setara ini.

B.3 Evolusi Transport & QUIC

QUIC berjalan di atas UDP karena: (1) **deployability** — middlebox internet (NAT, firewall) hanya meloloskan TCP/UDP, protokol IP baru akan banyak di-drop; (2) **menghindari kekakuan kernel OS** — TCP ada di kernel dan sulit diubah cepat, sedangkan di atas UDP QUIC bisa mengimplementasikan seluruh logic transport sendiri di user-space dan berevolusi lewat update aplikasi/library; (3) **kontrol penuh** — UDP tidak memaksakan model reliability/ordering, sehingga QUIC bebas merancang multiplexing sesuai kebutuhan modern.

Solusi HOL blocking: TCP hanya mengenal satu byte stream per koneksi, sehingga satu segmen hilang memblokir seluruh stream setelahnya (termasuk data resource lain yang di-multiplex, seperti HTTP/2 di atas TCP). QUIC mengimplementasikan multiplexing di level stream native dengan buffer reliability independen per stream: jika satu paket stream A hilang, hanya stream A yang menunggu retransmisi, stream lain yang sudah lengkap tetap langsung diserahkan ke aplikasi.

B.4 Skenario Bind Explicit pada Client

Default-nya OS memilih port ephemeral otomatis saat client pertama mengirim data. `clientSocket.bind('', 5432)` eksplisit memaksa socket memakai port 5432 tetap pada semua interface lokal, bukan port ephemeral acak.

Apakah server masih bisa membalas: Ya, karena balasan UDP murni bergantung pada alamat pengirim yang tercatat di header paket yang diterima lewat `recvfrom()`; server tinggal `sendto()` balik ke alamat itu, berfungsi identik dengan port ephemeral biasa.

Potensi kendala 2 instansiasi bersamaan: Port 5432 bersifat eksklusif per host; jika instansiasi client kedua juga mencoba bind ke port yang sama, `bind()` akan gagal dengan `OSError: Address already in use`, karena port sudah dipakai instansiasi pertama. Berbeda dari implicit bind, di mana setiap instansiasi otomatis mendapat port ephemeral berbeda dari OS sehingga banyak instansiasi bisa berjalan bersamaan tanpa konflik.